

衛星データ解析技術研究会 技術セミナー（応用編）

第4回 3D技術②

点群・3DGSのWeb可視化

2026年7月17日（金）

到達目標：3Dデータのフォーマットと処理パイプラインを説明できる／deck.glで点群をWeb地図上に表示できる／3DGSの現状と入口を把握する

本日のタイムテーブル（140分）

時間	内容
10分	イントロ・3Dデータフォーマット比較（3D Tiles / glTF / Potree / COPC）
15分	CesiumJS・Re:Earth の位置づけと活用事例（文化財・インフラ点検）
35分	ハンズオン①：CloudCompare → PotreeConverter → Web公開
10分	休憩
30分	ハンズオン②：deck.gl PointCloudLayer で点群を地図上に表示
15分	3DGS概説（フォーマットの現在地とCesiumJS統合の動向）
15分	振り返り・Q&A

ハンズオンの詳細手順は `handson/` 以下のREADMEにまとめてあります。本編では何を作るのか・なぜその構成なのかを中心に扱います。

前回までのつながり

- 第1～2回：**2Dのタイル配信**
 - タイルピラミッド・PMTiles・Nginxによる静的配信
- 第3回：**スマホセンサーとAR**
 - Geolocation / DeviceOrientation、カメラ映像への重ね表示
- 第4回（今日）：**3Dデータそのものを扱う**
 - 点群という「巨大な点の集まり」をどう配信・描画するか

2Dタイルで学んだ考え方（階層化して必要な分だけ読む）は、3Dでもそのまま生きる。タイルピラミッドの3D版が今日の主役のひとつ「octree（八分木）」。

1. 3DデータのフォーマットとWebGIS

Webで3Dを扱うときの根本的な問題

- LiDARスキャン1回で 数十万～数億点 の点群が生まれる
 - LASファイルで数GB規模になることも珍しくない
 - ブラウザに全部読み込むのは現実的でない
- 2Dタイルと同じ発想が必要になる

2Dの世界	3Dの世界
タイルピラミッド（z/x/y）	octree（八分木）による空間分割
ズームレベルに応じて解像度を切替	カメラ距離に応じてLOD（詳細度）を切替
PMTiles / MVT	3D Tiles / Potree / COPC
Range Request で必要な部分だけ取得	同じく必要なノードだけ取得

主要フォーマットの整理（1/2）：交換用と配信用

「編集・交換のためのフォーマット」と「Web配信のためのフォーマット」を区別すると見通しが良くなる。

分類	フォーマット	特徴
交換用（点群）	LAS	LiDAR業界標準。座標値・強度・色などを格納。非圧縮
交換用（点群）	LAZ	LASの可逆圧縮版。1/5～1/10程度に
交換用（汎用）	PLY	点群・メッシュ両対応の汎用形式。3DGSの出力にも使われる
交換用（メッシュ）	glTF / GLB	「3DのJPEG」と呼ばれる汎用3Dモデル形式
配信用	3D Tiles	Cesium発のOGC標準。glTFをタイル化して階層配信
配信用	Potree形式	点群専用。octree構造で分割済み
配信用	COPC	LAZ 1.4にoctree索引を埋め込んだ単一ファイル

第1回のPMTilesと同様、COPCも「単一ファイル + Range Request」という設計思想。COG（第5回で扱う）の点群版にあたる系譜。

主要フォーマットの整理（2/2）：使い分けの目安

やりたいこと	選択肢
スキャンした点群を手元で確認・編集する	LAS / PLY + CloudCompare
点群をWebで公開する（専用ビューア）	Potree形式 + Potreeビューア
点群をWebで公開する（静的ファイル1個で）	COPC
建物・都市モデルなどメッシュを配信する	3D Tiles（中身はglTF）
単体の3Dモデルを埋め込む	glTF / GLB

- **Potree形式**：`metadata.json` + `octree.bin` + `hierarchy.bin`（PotreeConverter 2系）
 - **3D Tiles**：`tileset.json` が階層を記述。**PLATEAU**（国交省の3D都市モデル）の配信形式として国内でも実質標準
- 3D Tilesの読み込みは `handson/04_cesium_3dtiles/` に補足資料あり

3D Tilesをもう少しだけ

- Cesium社が策定し、**OGC Community Standard** として標準化
- ルートの `tileset.json` から子タイルを辿る階層構造
- 各タイルの中身は glTF（3D Tiles 1.1 以降）

```
// tileset.json の骨格（抜粋イメージ）
{
  "root": {
    "boundingVolume": { "region": [...] },
    "geometricError": 500,
    "content": { "uri": "building.glb" },
    "children": [ ... ]
  }
}
```

`geometricError` がLOD切替の閾値。「カメラから見てこの誤差が目立つ距離まで近づいたら子タイルを読む」という仕組み。第1回で見たタイルピラミッドのズームレベル切替と同型の発想。

2. レンダリングエンジンの位置づけ

WebGLベースの3D GISエンジン比較

エンジン	得意分野	特徴
CesiumJS	地球儀・3D Tiles	3D Tilesの本家。地形・時間軸まで扱える
deck.gl	データ可視化レイヤー	地図タイルと重ねられる。点群はPointCloudLayer
Potree	点群専用ビューア	巨大点群の表示に特化。Three.jsベース。計測ツール内蔵
Three.js	汎用3D	GISではないが、上記の多くの基盤的存在
Re:Earth	ノーコードWebGIS	CesiumJSベース。国産OSS。プラグインで拡張

- CesiumJS：「**地球の上に載せる**」前提の設計。座標系・地形・カメラ制御が最初から地理空間仕様
- deck.gl：**「**地図の上にレイヤーを重ねる**」**発想。第2回までのMapLibreの延長線上で使える
- 今日はPotree（ハンズオン①）とdeck.gl（ハンズオン②）を使う

活用事例：なぜ点群をWebで見せたいのか

文化財

- 3D計測によるアーカイブ（劣化前の記録、修復の基礎資料）
- 現地に行けない人への公開、教育利用

建築・BIM

- 竣工時点群と設計モデル（BIM）の照合
- 図面のない古い建物の改修前の現況把握

インフラ点検

- 橋梁・トンネル・法面のスキャンによる変状把握
- 定期点検結果の時系列比較

共通する要求は「**現場に行かずに・複数人で・ブラウザだけで確認したい**」。関係者全員に専用ソフトのインストールを求めるのは現実的でない場面が多い。

リモートセンシングとの接点

- 航空LiDAR測量の成果は点群（LAS/LAZ）で提供される
- 衛星・航空写真からのSfM（Structure from Motion）でも点群が生成される
- 第5回で扱うDEM/DSMの多くは、**点群を内挿・ラスタライズしたものが起点**

今日の内容は「ラスタライズされる前の生データを直接扱う」回とも言える。

静岡県の「VIRTUAL SHIZUOKA」のように、県単位で点群をオープンデータ公開する事例もある（G空間情報センターで配布）。

ハンズオン①

CloudCompare → PotreeConverter → Web公開

手順書：[handson/01_cloudcompare/](#) および [handson/02_potree/](#)

ハンズオン①で作るもの



第2回でやった「PMTilesをNginxで静的配信」と構図は同じ。変換さえ済めば、あとは静的ファイルを置くだけ。

入力データ：モバイルLiDARスキャン

- iPhone 12 Pro / 2020年以降のiPad Pro等には、dToF式で約5mまで計測可能なLiDARセンサーが搭載されている
- Scaniverse・3d Scanner App・RealityScan・Polycam 等のアプリで点群を取得できる
- 本日は事前スキャン済みのLASファイルを配布（自分のスキャンデータの持ち込みも歓迎）

Scaniverseでの取得の要点：

- 新規データ作成＞メッシュ、スキャンモードは「スピード」または「エリア」で十分（後段で間引くため）
- エクスポートは **LAS** を選択（PLYも可だが今日はLASで統一）

重要：ScaniverseのLAS出力は**UTM投影座標系**で記録される（座標値がX=62万、Y=371万のような大きな絶対値になる）。これがハンズオン②で効いてくるので覚えておく。

CloudCompareでの前処理（概要）

CloudCompareはGPLライセンスの点群処理FOSSデファクト。今日やる工程は4つ。

工程	メニュー	目的
読み込み確認	Open LAS file ダイアログ	点数・座標系・Global Shiftの確認
クリッピング	Tools > Segmentation > Cross Section	対象範囲の切り出し
ノイズ除去	Tools > Clean > SOR filter	孤立点の統計的除去
サブサンプリング	Edit > Subsample（Space方式）	Web表示向けに点数を削減

- 読み込み時の **Global shift/scale** ダイアログ：大きな座標値を内部的にシフトして精度落ちを防ぐ仕組み。保存時は「シフトを保持」して書き出す
- 間引きの目安：**数十万点程度**。deck.glでは読み込む点数がそのまま描画負荷に直結する

PotreeConverterでの変換（概要）

```
PotreeConverter input.las -o ./output --generate-page index
```

- `--generate-page index` でビューア付き `index.html` も生成される

```
output/  
├─ metadata.json    # メタデータ（バウンディングボックス、属性定義等）  
├─ hierarchy.bin    # octreeの階層構造  
├─ octree.bin       # 点群データ本体  
└─ index.html       # ビューア付きHTMLページ（+ libs/ 等）
```

- Windows / Linux向けバイナリが提供されている。**macOSはDockerでLinuxバイナリを使う**（Dockerfileと注意点はREADMEに記載。Appleシリコンは `--platform linux/amd64` 指定）
- 圧縮オプションはないため、実運用ではサーバー側のBrotli/gzip圧縮を併用するとよい（GitHub Pagesは自動で圧縮配信）

ローカル確認と公開

`file://` での直接オープンは**不可**（fetchが失敗する。第1回のPMTilesと同じ理由）。

ローカル確認の選択肢：

- `python3 -m http.server 8000`
- npm の `http-server`
- VSCode拡張「Live Server」

公開先：

公開先	備考
GitHub Pages	置くだけ。1ファイル100MB制限に注意
Nginx（自前サーバー）	第2回の構成にディレクトリを足すだけ

ここまでで「スキャン → Web公開」のパイプラインが完成。

休憩（10分）

再開後：deck.glで点群を「地図の上に」置く

ハンズオン②

deck.gl PointCloudLayer で点群を地図上に表示

手順書：[handson/03_deckgl/](#)（完成版アプリ同梱）

Potreeとdeck.glの役割の違い

	Potree	deck.gl
主目的	点群単体をじっくり見る	地図・他レイヤーと組み合わせる
背景地図	なし（点群のみ）	地図タイルと統合
巨大点群	octreeで数億点も可	メモリに載る規模（数十万点目安）が現実的
座標の扱い	元の座標系のままでよい	地図と整合させる必要がある

deck.glで地図に重ねるには、**点群の座標を地図側の座標系と対応づける**必要がある。ここがハンズオン②の山場。

山場：座標系の対応づけ

前提の整理：

- ScaniverseのLAS出力は**UTM座標系**（メートル単位の大きな絶対値）
- deck.gl の `COORDINATE_SYSTEM.METER_OFFSETS` は「`coordinateOrigin`（経緯度）からのメートルオフセット」として座標を解釈する
- UTM座標をそのまま渡すと「原点から500km東、3720km北」と解釈され、**何も表示されない**

対処：**UTM絶対座標から基準値を差し引いて、スキャン地点付近を原点とする相対座標に変換する**

```
// LASヘッダのoffset値を差し引いてローカル化する例
points[i].position[0] -= las.xOffset;
points[i].position[1] -= las.yOffset;
points[i].position[2] -= las.zOffset;
```

差し引く値は点群のバウンディングボックス中心付近を使う。`pdal info` やCloudCompareのプロパティ表示で確認できる。

LASをブラウザで読む：自前パーサー方式

deck.glのエコシステムにはLAS用ローダー（loaders.gl）も存在するが、**CDNバンドル同士ではdeck.gl内蔵の画像ローダーと競合することがある**。今日は `DataView` でLASヘッダを直接読む自前パーサー方式を採用する。

```
// LAS 1.2ヘッダの要点（リトルエンディアン）
const offsetToPoints = view.getUint32(96, true); // 点レコードの開始位置
const pointFormat     = view.getUint8(104);      // Point Format (2/3ならRGBあり)
const numPoints       = view.getUint32(107, true); // 点数
const xScale          = view.getFloat64(131, true); // スケール (x = raw * scale + offset)
const xOffset         = view.getFloat64(155, true); // オフセット
```

- 座標は `int32のraw値 × scale + offset` で復元する（LAS仕様）
- RGBはPoint Format 2なら各レコードの20バイト目、Format 3なら28バイト目から16bit×3
- 副産物として、**LASのバイナリ構造そのものの理解**が得られる（本セミナーの理念とも合致）

読み込むスクリプトはdeck.glのCDNバンドルひとつだけ。パーサー全文はhandson/03_deckgl/app/index.htmlに同梱。

deck.glでの表示構成

```
new deck.DeckGL({
  initialViewState: {
    longitude: SCAN_ORIGIN[0], latitude: SCAN_ORIGIN[1],
    zoom: 22, pitch: 45, maxZoom: 24
  },
  controller: true,
  layers: [
    new deck.TileLayer({           // 背景地図：OSMラスタータイル
      data: 'https://c.tile.openstreetmap.org/{z}/{x}/{y}.png',
      renderSubLayers: (props) => new deck.BitmapLayer(...)
    }),
    new deck.PointCloudLayer({    // 点群本体
      data: points,
      coordinateOrigin: SCAN_ORIGIN, // スキャン地点の経緯度
      coordinateSystem: deck.COORDINATE_SYSTEM.METER_OFFSETS,
      getPosition: (d) => d.position,
      getColor: (d) => d.color,
      pointSize: 3
    })
  ]
});
```

- **pitch** を上げないとZ方向（高さ）が見えない

つまづきポイント（先回りメモ）

症状	原因と対処
点群が表示されない	UTM絶対座標のまま渡している。offset差し引きを確認。 <code>console.log</code> で最初の点の座標を見る
点が灰色一色	Point Format 0/1（RGBなし）。エクスポート設定を確認
点群が地図とずれる	<code>SCAN_ORIGIN</code> の経緯度が実際のスキャン地点と合っていない。Scaniverseの位置情報表示で確認
ページが固まる	点数が多すぎる。CloudCompareの間引きに戻る（数十万点目安）
fetchが失敗する	<code>file://</code> で開いている。http-server等を経由する

3. 3D Gaussian Splatting (3DGS) 概説

3DGSとは何か

2023年のSIGGRAPH論文を起点に急速に普及した、写真群から学習される3Dシーン表現。

- 点群：シーンを「色付きの点」の集合で表す
- 3DGS：シーンを「位置・大きさ・向き・不透明度・視点依存の色を持つ楕円体（ガウシアン）」の集合で表す

	点群（LiDAR）	3DGS
取得方法	レーザー測距	複数枚の写真から最適化（学習）
幾何精度	高い（測量に使える）	見た目優先（寸法保証なし）
見た目	点の隙間が見える	写實的。反射・光沢も再現
CloudCompareで編集	可	通常の点群としては扱えない

「測る」なら点群、「見せる」なら3DGS、という住み分けが現時点の目安。NeRFと同じく「写真からの3D復元」だが、明示的なプリミティブの集合なのでリアルタイム描画に向く。

3DGSのフォーマット事情：まだ標準化の途上

形式	由来・特徴
PLY	原論文実装の出力。球面調和係数まで含み 非常に大きい
.splat	コミュニティ発の簡易圧縮形式。位置・スケール・色・回転を量子化。Webビューアで広く使われる
.ksplat	Three.js系ビューア（GaussianSplats3D）の圧縮形式
SPZ	Niantic（現Niantic Spatial）が公開。PLY比 約1/10。Scaniverseのエクスポートにも採用
glTF拡張	Khronosでガウシアンスプラット拡張の策定が進行中。 glTFに載る = 3D Tilesに載る ため本命視される

現状の整理：

- PLY派生・.splat・SPZ・glTF拡張が**並立している過渡期**
- 「点群にとってのPotreeConverter」に相当する定番変換ツールは確立途上
- 方向性は明確：**glTF/3D Tilesのエコシステムへの合流**

数年前のベクタータイル（MVT標準化前夜）に似た状況。標準が固まる前でも「取得 → 変換 → 階層化 → 静的配信」という構図は共通しているので、今日学んだ考え方はそのまま応用できる。

3DGSを体験・編集する最短ルート

学習には本来GPUが必要だが、アプリ・SaaSがその部分を肩代わりしてくれる。

取得（スマホ・SaaS）

サービス	特徴
Scaniverse	Splatモード搭載。端末内処理。SPZ等でエクスポート可
Luma AI	動画アップロードでクラウド処理。Web埋め込み可
Polycam	写真測量・LiDAR・3DGSを統合

閲覧・編集（ブラウザ）

- **SuperSplat**（PlayCanvas製・OSS）：ブラウザ上で3DGSの閲覧に加え、不要スプラットの削除・切り抜きなどの編集ができる。点群におけるCloudCompareに近い立ち位置
- 試し方の手順は `handson/05_3dgs/` に補足資料あり

3DGSとWebGIS：統合の動向

3DGSを「地図の上に置く」動きが2024年以降活発化している。

- **CesiumJS**：3D Tilesにガウシアンスプラットを載せる取り組みを進めている
 - 巨大な3DGSシーンをタイル分割・ストリーミングする方向性
 - glTF拡張の標準化と歩調を合わせた動き
- **deck.gl / Three.js**：コミュニティ製のsplatレンダラーが複数存在

見立て：

- 幾何精度が要らない「見せる」用途（観光・広報・現場共有）から実用化が進む
- 測量・点検など精度が必要な領域では点群が引き続き主役
- 両者を同じ**3D Tiles基盤**の上で重ねて配信するのが到達点になりそう

この分野は動きが速い。本スライドの記述は2026年前半時点の状況として読むこと。

振り返り：今日の到達点

- フォーマット：交換用（LAS/LAZ/PLY/glTF）と配信用（3D Tiles/Potree/COPC）を区別できる
- パイプライン：スキャン → CloudCompare → PotreeConverter → 静的配信、を自分の手で完走した
- 座標系：UTM絶対座標とローカルオフセットの関係を理解し、deck.glで点群を地図に載せた
- **LASの中身**：ヘッダをDataViewで読み、`raw × scale + offset` の構造を確認した
- **3DGS**：点群との使い分けと、glTF/3D Tilesへの合流という潮流を把握した

一貫していたのは第1回からの構図：

巨大なデータを、階層化して、静的ファイルとして配信する。

補足資料（handson/ 以下）

ディレクトリ	内容
01_cloudcompare	CloudCompareでの前処理の全手順
02_potree	PotreeConverterの導入（Docker含む）と変換・公開
03_deckgl	deck.glアプリ完成版と座標系解説
04_cesium_3dtiles	【補足】CesiumJSの試し方と3D Tiles（PLATEAU）の読み込み
05_3dgs	【補足】3DGSの取得・閲覧・編集・フォーマット詳説

04と05は本日のハンズオンでは扱わないが、自習できるよう独立した資料として用意している。

次回予告：第5回 高さデータ①

- リモートセンシングによる高さデータ（DEM/DSM）の生成原理
 - InSAR・LiDAR・SfM
- GeoTIFF / COG / LAS / LAZ のフォーマット構造
- GDALによる変換パイプライン：座標変換 → クリッピング → Terrain RGBタイル化

今日扱った点群は、内挿・ラスタライズを経てDEM/DSMになる。**第4回と第5回はデータの連続体として見てほしい。**

7/24（金）同時刻・同会場。

お疲れさまでした

質疑応答

ハンズオンで完走できなかった方は [handson/](#) 以下のREADMEに全手順があります

リポジトリ：github.com/alt9800/2026-RemoteSensingSeminar